Cat GPT

Project Deep Dive & Engineering Handoff

Architecture, runtime flows, data ownership, agent behavior, frontend, deployment, and operational guidance

Prepared from the project source tree. Secret values and local credentials are intentionally excluded.

Contents

- 1. Executive overview and system map
- 2. Repository structure and file responsibilities
- 3. Runtime architecture and request lifecycles
- 4. Authentication, identity, and account isolation
- 5. LangChain/LangGraph agent and model behavior
- 6. Tools and external services
- 7. RAG and document ingestion
- 8. Persistence, databases, and migrations
- 9. FastAPI endpoints and SSE contract
- 10. Next.js frontend and state management
- 11. Configuration and deployment
- 12. Security, reliability, known constraints, and future work
- 13. Tests and developer workflows

1. Executive overview and system map

Cat GPT is a multi-user conversational assistant with a Next.js browser interface and a FastAPI service. The assistant runtime is implemented with LangChain and LangGraph, backed by Groq models, LangGraph SQLite checkpoints, a separate SQLite memory/chat store, and FAISS retrieval for uploaded knowledge.

The repository also retains a Streamlit interface in app.py. The Next.js/FastAPI application is the deployed web architecture, while Streamlit is a legacy/local interface that shares parts of the agent and filesystem layout. Keep this dual-surface fact in mind when changing shared modules.

High-level request flow

```
Browser (Next.js / forntend)
-> JSON/SSE HTTP with bearer session token
FastAPI (api/main.py)
-> validates request + account scope
LangGraph (agent/graph.py)
-> assistant model <-> LangChain ToolNode
Tools / RAG / Groq / Pollinations / SQLite / FAISS
-> event stream back through FastAPI SSE
Browser updates the current chat and media library
```

The central design invariants are: preserve the existing framework-driven agent graph, keep all user-owned data scoped by authenticated identity, stream real backend events rather than simulated progress, and keep the API/UI contract stable.

Current model/provider overview

- Text LLM: Groq ChatGroq model IDs configured by GROQ_MODELS (preferred list, maximum six) or GROQ_MODEL (legacy single model). Failover is handled in assistant_node for rate limits and unavailable models.
- Vision LLM: a separate GROQ_VISION_MODEL is selected when the incoming conversation contains image content.
- Image generation: Pollinations hosted Flux via HTTP in agent/tools.py. POLLINATIONS_IMAGE_MODEL defaults to flux; service access/free availability is controlled by the external provider.
- Embeddings: sentence-transformers/all-MiniLM-L6-v2 through HuggingFaceEmbeddings; run locally in the Python service and used for FAISS index operations.
- Voice: Groq Whisper model for transcription, Groq PlayAI TTS with gTTS fallback.

2. Repository structure and file responsibilities

Path	Responsibility
api/main.py	FastAPI app, CORS/auth middleware, auth/thread/media/chat/upload/voice routes, streaming event mapping.
api/auth.py	Google ID-token verification; password validation/hash/verification; signed session-token lifecycle.
api/store.py	SQLite accounts and per-user UI-thread state: pins, archive, rename, deletion, checkpoint cleanup.
api/config.py	API filesystem paths, CORS origins, upload limits, auth and rate-limit settings.
api/schemas.py	Pydantic request/response models and validation constraints.
api/rate_limit.py	In-process sliding-window attempt limiting for sign-in and registration.
api/streaming.py	SSE JSON formatting and extraction of assistant text, activity, and generated image markers.
agent/graph.py	LangGraph state, Groq models, prompts, model failover, image parsing, thread namespace, run/stream orchestration.

Path	Responsibility
agent/tools.py	LangChain tools and event context: calculator, web search, RAG, image, events, user memory.
agent/llm_recovery.py	Recovery from malformed model tool-call output; the recovered call re-enters normal tool routing.
agent/voice.py	Groq Whisper transcription, Groq PlayAI speech generation, gTTS fallback.
agent/quiz.py	Study-guide generation using ChatPromptTemplate and configured text model.
agent/image.py	Older/minimal Pollinations URL helper; current account image tool is in agent/tools.py.
database/db.py	SQLite tables and helpers for legacy thread/message data and per-user key/value long-term memories.
rag/ingest.py	Secure file/URL loading, OCR/document parsing, chunking, embeddings, FAISS updates, metadata registry, progress.
rag/search.py	FAISS semantic search and opening-chunk preview for broad attached-document requests.
utils/security.py	File/path/input validation helpers used by ingestion and upload flows.
app.py	Streamlit UI, legacy account/thread interaction surface, streaming display, upload and message flows.
forntend/app/page.tsx	Main Next.js chat/login/images/library interface, message actions and Google Identity Services button.
forntend/lib/useAuth.ts	Client authentication state, token storage, email/Google auth, session restoration/logout.
forntend/lib/useChat.ts	Chat/thread state, account-switch reset, attachments, streaming, image object URLs and voice.
forntend/lib/api.ts	HTTP/SSE client, bearer headers, response mapping and typed API error handling.
forntend/lib/types.ts	Frontend and backend DTO/type definitions.
forntend/components/Sidebar.tsx	Chat/navigation/sidebar, account menu, thread management and logout.
forntend/components/ChatInput.tsx	Message composer and attachment/input controls.
forntend/components/HomeHero.tsx / QuickActions.tsx	Empty-chat landing and prompt shortcuts.
forntend/components/CatLogo.tsx	Cat GPT logo asset rendering/fallback.
forntend/components/ui/*	Reusable button and icon-button primitives.
forntend/app/layout.tsx / globals.css	Next.js document metadata and global styling/theme.
tests/	Focused unit/API tests for agent graph, auth, calculator, database, image/library, fallback, RAG, and file security.
scripts/migrate_existing_data.py	One-off assignment of legacy unowned UI/checkpoint/RAG/memory rows to a selected account.
requirements.txt / runtime.txt	Python dependency and runtime configuration.
render.yaml / cat-gpt-deploy/*	Render deployment Blueprint and companion deployment guide; may be stale relative to Railway deployment.
.streamlit/config.toml	Streamlit theme/runtime configuration.

Generated data (SQLite databases, FAISS files, uploads, generated images, caches, node_modules, .next, and virtual environments) is runtime/development material rather than source documentation. Environment files are deliberately omitted from this report because they contain credentials.

3. Runtime architecture and request lifecycles

Application startup

FastAPI creates the app with an async lifespan. The UI database initializer creates/migrates user and per-user thread bookkeeping tables. The LangGraph checkpoint connection is separately created as the graph is built/run. Startup configuration should fail explicitly when required authentication/model settings are missing.

Chat turn

- The browser posts a SendMessageRequest to POST /threads/{thread_id}/messages/stream with text, optional image payloads, and references to uploaded files.
- The auth middleware validates the bearer token and stores user_id in request.state. The endpoint rejects blank/oversized prompts, too many images, empty requests, and file paths outside that account's uploads.
- A synchronous event_stream generator invokes stream_agent(user_id, messages, thread_id). Starlette moves sync streaming work off the event loop.
- LangGraph loads the user-namespaced checkpoint state, adds new messages through the messages reducer, invokes assistant_node, and routes tool calls via tools_condition to ToolNode.
- Graph emits updates and custom events. FastAPI maps them to text, activity, image, done, or error SSE events. The frontend parser incrementally updates the assistant bubble and activity state.

Why thread scoping matters

get_thread_config namespaces raw thread IDs using the authenticated user ID, so identical frontend thread IDs cannot collide across accounts. APIs additionally filter UI state by user_id and validate media paths against per-user directories.

Agent context

agent.tools uses ContextVars for active_user_id and active_source_filter. The graph sets them before invocation/streaming and clears them in finally blocks. Tools read these values to scope user memories, generated images, and retrieval to the active user/attached files. Do not replace them with mutable global request variables.

4. Authentication, identity, and account isolation

Email/password

Registration validates normalized email, name, and password limits, hashes with Argon2, rejects duplicate case-insensitive email, and returns a signed session token. Login uses a generic invalid-credentials response and performs a dummy Argon2 verification for unknown/Google-only accounts to reduce account enumeration. Password endpoints are protected by the same bearer middleware.

Google

The frontend loads Google Identity Services and renders its official credential button. Its callback posts the ID token to POST /auth/google. The backend verifies the signature/audience against GOOGLE_CLIENT_ID, requires Google’s email_verified claim to be true, and resolves the user by stable Google subject first, then verified email. A matching existing email/password account is reused and the Google subject is attached to preserve that user_id/data; a new verified identity creates an account automatically. If the email is already tied to a different Google subject, the service refuses to merge identities.

Google account selection/password/2FA happens on Google’s hosted UI. Cat GPT must never receive a Gmail password; it receives only a signed credential. Google and Cat GPT passwords remain distinct.

Session token

The backend signs '<user_id>.<expiry>' with HMAC-SHA256 using SESSION_SECRET. Frontend stores the bearer token in localStorage and verifies it on startup via GET /auth/me. Unauthorized responses trigger the frontend unauthorized handler, which clears the local session.

Account-specific API access

AuthMiddleware protects every path except a small explicit allowlist (/health, Google auth, registration/login, API docs). CORS wraps auth middleware so that browser-visible 401 errors still receive CORS headers. Thread history, pins, archive state, uploads, FAISS metadata/search, memories, generated images, and media libraries must never cross user boundaries.

Rate limiting and residual production constraints

api/rate_limit.py uses in-memory sliding windows, which reset on process restart and are not shared across multiple Railway replicas. Behind proxies, request.client.host needs correctly configured proxy headers; otherwise IP limiting may aggregate users at the proxy address. Production scaling should move counters to shared storage.

5. LangChain/LangGraph agent and model behavior

Graph state and topology

AgentState contains messages: Annotated[list[BaseMessage], add_messages]. The reducer is essential: it appends node output to checkpointed state rather than overwriting the conversation. build_graph creates assistant and tools nodes, uses LangGraph ToolNode and tools_condition, and compiles with a SQLite checkpointer.

```
START -> assistant
assistant -- tool_calls --> tools -> assistant
assistant -- no tool call --> END
```

Model selection and fallback

get_text_model_candidates reads GROQ_MODELS first (comma-separated, deduplicated, at most six IDs), otherwise GROQ_MODEL. get_vision_model_name reads GROQ_VISION_MODEL. If any HumanMessage contains an image_url block, assistant_node uses the vision model; otherwise it tries configured text models in order. On rate-limit/model-unavailable errors it can switch to the next candidate and emit a model_fallback event. Bounded retries and visible wait events avoid indefinite hidden retries.

ChatGroq is created with max_retries=0 because retry/fallback is explicitly coordinated in assistant_node. Token ceilings are separate for text and vision, controlled by GROQ_MAX_TOKENS/GROQ_VISION_MAX_TOKENS. Invalid/missing setup is surfaced as an assistant configuration response rather than silently defaulted to an unrelated model.

System prompt and memory

build_system_prompt constructs Cat GPT instructions and injects long-term memories for the current active_user_id. Memories are not persisted as ordinary chat messages; they are independently stored and read from database/db.py. This keeps profile memory distinct from chat checkpoint history.

Input parsing and conversation operations

parse_input_messages converts API dictionaries to LangChain HumanMessage objects, validates image data, prepares compact image payloads for Groq, and resolves attached files. Thread IDs are namespaced by user; graph helpers load history, deduplicate already-saved inputs, remove the latest turn for edit/retry, and derive titles. run_agent is synchronous; stream_agent yields LangGraph (mode, chunk) tuples and closes its checkpoint connection reliably.

Tool call recovery

assistant_node catches malformed tool-call provider output and asks agent/llm_recovery.py to recover an allowlisted tool call. A recovered call is returned as an ordinary AIMessage tool_calls structure so LangGraph's normal tool routing still applies; it does not execute arbitrary model-produced code.

6. Tools and external services

Tool/service	Behavior and operational notes
calculator	Evaluates a constrained expression AST with operator allowlists and limits; it is not a Python REPL.
web_search	LangChain TavilySearch; requires TAVILY_API_KEY, validates query and emits started/completed/error activity.
rag_search	Calls FAISS search with active attached-source and account context; broad attached-file requests can use get_document_preview fallback.
generate_image	Calls Pollinations Flux URL over requests with timeout, validates response image type/content, saves PNG under hashed account directory, returns marker consumed by SSE parser.
events	create_event/list_events/delete_event tools store user-associated event records in the app database; tool reads/writes must carry active_user_id.
memory	save_user_memory/get_user_memories tools use the active user's memory namespace.
speech-to-text	Groq Whisper (whisper-large-v3-turbo); requires GROQ_API_KEY.
text-to-speech	Groq PlayAI TTS first, then gTTS fallback if Groq call fails.
image understanding	Groq vision model processes attached image blocks; separate from image generation.

emit_tool_event sends custom LangGraph stream events through get_stream_writer. It tolerates calls outside streaming contexts so tools remain usable from Streamlit/tests. Tool activity is observational: it does not choose tools or perform graph routing.

Image generation is an external service dependency. 'No API token required' does not guarantee permanent free service, uptime, rate limits, acceptable latency, or privacy guarantees; review provider policy before production use. Generated files are user-scoped locally and are not uploaded to persistent object storage by the current architecture.

7. RAG and document ingestion

Ingestion flow

- Upload path: FastAPI validates extension, non-empty bytes, configured size, and count; stores file under UPLOAD_DIR/<opaque-user>/<safe-thread>/ with generated prefix; then invokes ingest_file in a thread pool.
- ingest_file sanitizes the path and validates file security before parsing. Supported loaders include PDF, text, CSV, DOCX, and image OCR (Pillow/Tesseract). URL ingestion uses HTTP requests and BeautifulSoup.
- Text is split with RecursiveCharacterTextSplitter. HuggingFaceEmbeddings (all-MiniLM-L6-v2) embeds document chunks. FAISS index updates and the pickle metadata registry are serialized under FileLock to avoid corrupt concurrent writes.
- Progress events report security validation, loader/indexing stages, batches, and heartbeat while first loading model weights. API maps each upload to its own result/error, so one bad file need not fail the entire batch.

Search and account isolation

FAISS stores vectors under data/faiss; metadata records source, chunks, and user_id. search_pdf validates query/k and filters by sources when attachment scope is supplied. get_document_preview is for generic questions about attached files and uses source order instead of similarity ranking. Validate that all source metadata is account-filtered consistently before using a shared index in multi-user production.

Security and persistence caveat

FAISS.load_local uses allow_dangerous_deserialization=True for the project's own local index pickle. The index and metadata must therefore be treated as trusted application files: never accept a client-supplied path/index or load untrusted pickle content. Local disk is not durable on many hosted container tiers.

8. Persistence, databases, and migrations

Store	Owner / data
data/ui.sqlite (UI_DB_PATH)	api/store.py account records (google_id, email, name, password_hash), plus per-user pin/delete/rename/archive tables.
data/checkpoints.sqlite (CHECKPOINT_DB_PATH)	LangGraph SqliteSaver checkpoints containing conversation state keyed by user-namespaced thread IDs.
data/assistant.db (database/db.py)	Legacy/agent threads/messages and long-term per-user memories. Uses SQLite WAL wrapper.
data/faiss/*	FAISS vector index and metadata pickle registry for ingested sources/chunks.
data/upload/*	Uploaded files, scoped by user/thread for API uploads.
data/generated_images/*	Generated PNGs scoped to a SHA-256 hash of user_id.

Do not merge these stores casually. They have different schemas, callers, migration histories, and persistence lifetimes. database/db.py provides WAL mode, initialization/migration of memories, thread/message CRUD, and per-user memory upserts. api/store.py creates and transactionally migrates auth and UI tables, including legacy nullable user_id rows.

scripts/migrate_existing_data.py is an explicit one-off tool assigning legacy unowned UI state, checkpoint thread IDs, RAG metadata, and memories to one selected user by email. Back up all databases and FAISS metadata before running it. Review the script against the current schema/deployment before executing on production.

Hosted storage

SQLite/FAISS/uploads/images assume writable filesystem paths. A container filesystem can be ephemeral across restarts/redeploys. Railway production should attach a persistent volume and configure DATA_DIR, UI_DB_PATH, UPLOAD_DIR, and CHECKPOINT_DB_PATH under that volume. A multi-replica backend cannot safely share arbitrary local SQLite/FAISS state without deliberate concurrency/storage design.

9. FastAPI endpoints and SSE contract

Endpoint families

- Auth: GET /auth/me; POST /auth/google; POST /auth/register; POST /auth/login; POST /auth/set-password.
- Thread actions: POST/GET /threads; GET /threads/{id}/messages; pin/unpin; archive/unarchive; rename; edit latest turn; delete; export Markdown.
- Chat: POST /threads/{id}/messages/stream.

- Uploads/media: POST /files/upload; POST /images/generate; GET /images; GET /library/media; authenticated file/image fetch endpoints.
- Voice: POST /voice/transcribe; POST /voice/speak.
- Health/docs: GET /health is public; API docs are exposed according to the auth allowlist and deployment policy.

SSE event types

event	data shape	meaning
text	{text: string}	Assistant text delta/final output.
activity	{message: string}	Progress/tool/ingestion/model activity.
image	{url: string}	Generated image in account-protected image endpoint.
done	{}	Turn completed normally.
error	{message: string}	Turn failed; stream ends.

api/streaming.py extracts content from LangGraph 'updates' and 'custom' chunks and recognizes [GENERATED_IMAGE] tool markers. api/main.py emits SSE frames with JSON payloads. frontend/lib/api.ts parses event/data lines, ignores malformed/unknown events safely, and exposes a typed stream to useChat.

10. Next.js frontend and state management

Authentication surface

LoginScreen in page.tsx renders email/password sign-in and registration, toggles password visibility, and renders Google Identity Services when NEXT_PUBLIC_GOOGLE_CLIENT_ID is configured. useAuth owns token/session state and provides loading/errors. It posts credentials to the matching backend routes and stores the signed token only on success. The Google sign-in copy indicates verified-email auto-resolution and account creation.

Chat surface

useChat owns thread list/loading/error, active conversation/messages, in-flight SSE stream, uploads, image previews, recording/playback, and abort lifecycle. When the session token changes it cancels active work, clears previous account data/object URLs, and refreshes the new account’s threads. This reset is important to prevent old-account chats briefly appearing after account switch.

API helper and contracts

api.ts applies bearer auth from localStorage, maps DTO snake_case to UI camelCase, turns non-2xx responses into ApiError, invokes unauthorized callback on 401, handles authenticated image fetches, and parses SSE. DTO definitions in types.ts should remain synchronized with api/schemas.py.

Visible UI

page.tsx controls view/layout, chat message rendering, temporary assistant streaming indicator, message copy/edit/retry/listen actions, image-generation panel and Library view. Sidebar.tsx owns navigation, conversation actions, profile/export/logout menu. CatLogo loads a replaceable public logo with a fallback. ChatInput handles prompt and attachments.

Media URLs

Generated image endpoints require Authorization. The frontend must fetch protected images with bearer headers and create/revoke object URLs; a plain <img src> to an authenticated endpoint will not include a bearer token automatically. Review every new media display path for this distinction and revoke URLs on account/thread changes or unmount.

11. Configuration and deployment

Backend secret/config variables

- GROQ_API_KEY: required for Groq chat, vision, and Groq speech operations.
- GROQ_MODELS or GROQ_MODEL: text model list or single model; at most six configured candidates.
- GROQ_VISION_MODEL: required for image-understanding requests.
- GOOGLE_CLIENT_ID: backend audience check for GIS ID tokens; must match Vercel public client ID.
- SESSION_SECRET: high-entropy random HMAC key, unique per environment; rotating invalidates existing session tokens.
- FRONTEND_ORIGIN: comma-separated exact browser origins used by CORS, e.g. production Vercel origin and local dev origin if needed.
- TAVILY_API_KEY: required for Tavily web search; optional only if that feature is disabled/not used.
- TAVILY key is accessed through integration config; secret must remain server-only.
- POLLINATIONS_IMAGE_MODEL defaults to flux; no HF token is used by current image generation. HF_IMAGE_MODEL/HF_API_TOKEN may be stale legacy config.
- DATA_DIR, UI_DB_PATH, UPLOAD_DIR, CHECKPOINT_DB_PATH: set paths under persistent Railway volume if persistence is required.

Frontend variables (Vercel)

- NEXT_PUBLIC_API_BASE_URL: Railway backend public HTTPS URL (no trailing slash preferred).
- NEXT_PUBLIC_GOOGLE_CLIENT_ID: same Google OAuth Web client ID as backend.

Google Cloud configuration

Add the deployed Vercel origin (scheme + hostname, no path) under OAuth authorized JavaScript origins. Local development origin is separate. Google verification credentials are not the Cat GPT password. Never expose provider API keys using NEXT_PUBLIC_ variables.

Deployment template mismatch

The repository contains Render-oriented render.yaml and cat-gpt-deploy deployment notes, but the user's stated deployment target is Railway for FastAPI and Vercel for Next.js. Treat the Render blueprint/guide as reference/stale, not proof of current Railway setup. Verify Railway start command, port binding, persistent volume mount, environment variables, health route, and Vercel root directory forntend.

Deployment caveats

Heavy dependencies (torch, sentence-transformers, FAISS) increase build time, memory use, and cold starts. First RAG ingestion may download embedding weights. Configure an appropriate Railway memory/CPU plan and volume. Use HTTPS and restrict CORS to actual frontend origins. Never commit .env/.env.local or deploy logs containing secrets.

12. Security, reliability, known constraints, and future work

Security invariants

- Treat all client data as untrusted: validate prompt sizes, file extensions/sizes, base64 image payloads, thread IDs, and upload file paths.
- Use parameterized SQL and per-user predicates for every user-owned record.
- Keep Google token verification server-side; require correct audience and verified email before account matching.
- Never pass arbitrary filesystem paths from clients; resolve uploads under the authenticated account's root.
- Do not reintroduce a Python execution tool; arbitrary code execution via model tool calls was intentionally removed.
- Keep generated filenames opaque and validate basename/pattern before serving.
- Avoid logging tokens, passwords, raw .env values, or sensitive upload contents.

Reliability and limitations

The rate limiter is process-local, SQLite/FAISS are single-volume local persistence assumptions, and external providers can rate-limit or change free access. SSE requests can be interrupted by client navigation/disconnect; cancel/close resources and keep errors distinguishable from successful completions. Ingestion progress callback errors must not break ingestion. Image and voice fallbacks should not silently misrepresent which provider actually completed work.

Known maintenance considerations

- There are two frontends (Streamlit app.py and Next.js forntend) plus separate backend/data compatibility paths. Changes to shared graph/tools/RAG code may affect both; test the relevant caller(s).
- Some comments/config still refer to Hugging Face image generation although current agent/tools.py uses Pollinations Flux. Remove/update only when deliberately cleaning that scoped behavior.
- The small agent/image.py helper creates a Pollinations URL but account-scoped generation is implemented by the LangChain generate_image tool in agent/tools.py.
- RAG metadata/index currently use shared on-disk FAISS layout; validate user filtering carefully, especially for indexing, deletion, and shared host deployments.
- Current user requests have prioritized preserving architecture and avoiding unrelated refactors; keep that as the default.

Potential future evolution

For durable, horizontally scaled production, consider moving relational state and LangGraph checkpoints to managed PostgreSQL, storing uploads/generated images in object storage, and using a shared rate limiter/cache. Add explicit migration/backup/restore procedures and retention policies before moving. These are architectural proposals, not implemented current behavior.

13. Tests and developer workflows

Test module	Coverage focus
tests/test_agent.py	Graph construction, assistant/tool routing, thread/history and agent contract behavior.
tests/test_model_fallback.py	Configured Groq model order, provider fallback and retry behavior.
tests/test_auth.py	Password registration/login/session, Google credential rejection and identity behavior.

Test module	Coverage focus
tests/test_db.py	SQLite thread/message/memory schema and operation semantics.
tests/test_calculator.py	Calculator allowed expressions, bounds, and invalid inputs.
tests/test_rag.py	Document ingestion/search, index, metadata, and validation behavior.
tests/test_security.py	File/path/input validation and upload security.
tests/test_image_library.py	Account scoping for files/images, media endpoints, generated image provider handling.

Commands

```
Backend focused tests (PowerShell, repository root):
$env:PYTHONPATH = (Get-Location).Path
.\.venv\Scripts\pytest.exe tests\test_auth.py -q

Frontend type check (PowerShell, forntend directory):
npx tsc --noEmit

Frontend production build:
npm run build
```

Use targeted tests for touched modules first. Do not report a test as passing unless it was actually run in the current task. Treat environment configuration and actual deployed OAuth origins separately from source-level tests; successful local auth tests do not prove the deployed Google project is configured correctly.

End of project handoff.